Lecture_5

Windows Programming_II

Dr. Sara Mohamed

Building Graphical Interface Elements by Using the System. Drawing

Build graphical interface elements by using the System.Drawing namespace.

- The Windows Graphics Design Interface (also known as GDI+) classes can be used to perform a variety of graphics-related tasks such as working with text, fonts, lines, shapes, and images. One of the main benefits of using GDI+ is that it allows you to work with Graphics objects without worrying about the specific details of the underlying platform. The GDI+ classes are distributed among four namespaces:
  - System.Drawing
  - System.Drawing.Drawing2D
  - System.Drawing. Imaging
  - System.Drawing. Text
  - All these classes reside in a file named System.Drawing.dll

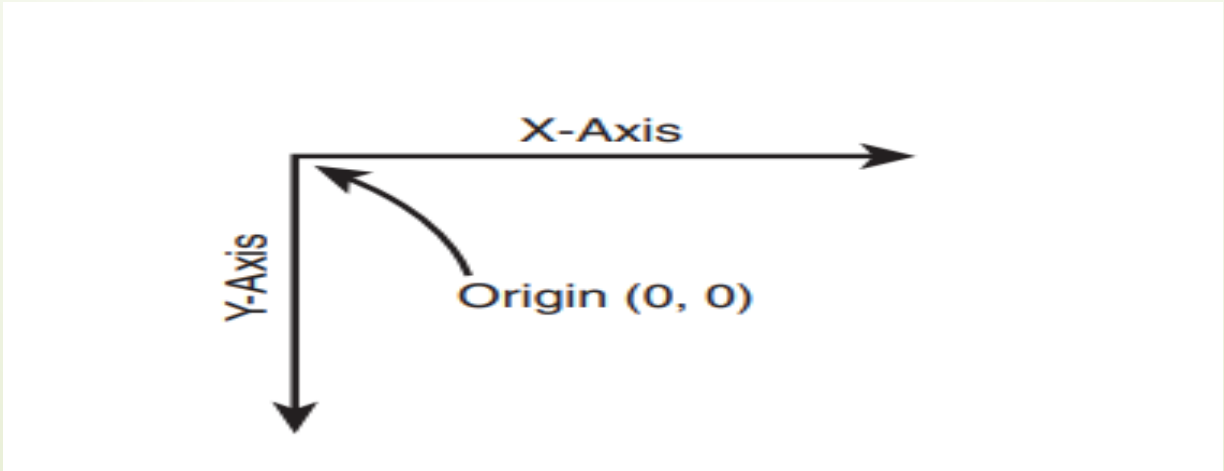
Graphics class

The Graphics class is one of the most important classes in the System.Drawing namespace. It provides methods for doing various kinds of graphics manipulations. The Graphics class is a **sealed** class and cannot be further inherited (unlike the Form class, for example). The only way you can work with the Graphics class is through its instances (that is, Graphics objects). A Graphics object is a GDI+ drawing surface that you can manipulate by using the methods of the Graphics class.

- When you have a Graphics object available, you have access to a drawing surface. You can use this surface to draw lines, text, curves, shapes, and so on. But before you can draw, you must understand the Windows forms coordinate system, which is discussed in the following section

Understanding the Windows Forms Coordinate System

- The Windows Forms library treats a Windows form as an object that has a two-dimensional coordinate system, Therefore, when you write text on the form or put controls on the form, the position is identified by a set of points.
- A point is a pair of numbers that is generally represented as (x, y) where x and y, respectively, denote horizontal and vertical distance from the origin of the form. The origin of the form is the top-left corner of the **client area** of the form. (**The client area** is the inner area of the form that you get after excluding the space occupied by title bar, sizing borders, and menu, if any.) The point of the origin is treated as (0, 0). The value of x increases to the right of the point of origin, and the value of y increases below the point of origin.



- Two structures are available to represent points in a program—Point and PointF. These structures each represent an ordered pair of values (x and y). Point stores a pair of int values, whereas PointF stores a pair of float values. In addition to these values, these structures also provide a set of static methods and operators to perform basic operation on points.

The structures defined in the System.Drawing namespace

System.Drawing Namespace STRUCTURES

Structure	*Description*
`CharacterRange`	Specifies a range of character positions within a string.
`Color`	Specifies a color structure that has 140 static properties, each representing the name of a color. In addition, it also has four properties—`A`, `R`, `G`, and `B`—which specify the value for the `Alpha` (level of transparency), `Red`, `Green`, and `Blue` portions of the color. A `Color` value can be created by using any of its three static methods: `FromArgb()`, `FromKnownColor()`, and `FromName()`.
`Point`	Stores an ordered pair of integers, x and y, that defines a point in a two-dimensional plane. You can create a `Point` value by using its constructor. The `Point` structure also provides a set of methods and operators for working with points.

Structure	*Description*
`PointF`	Specifies a `float` version of the `Point` structure.
`Rectangle`	Stores the location and size of a rectangular region. You can create a `Rectangle` structure by using a `Point` structure and a `Size` structure. `Point` represents the top-left corner, and `Size` specifies the width and height from the given point.
`RectangleF`	Specifies a `float` version of the `Rectangle` structure.
`Size`	Represents the size of a rectangular region with an ordered pair of width and height.
`SizeF`	Specifies a `float` version of the `Size` structure.

Drawing Text on a Form

- The Graphics class provides a **DrawString()** method that can be invoked on a Graphics object to render a text string on the drawing surface.
- Create a new project then Open the Properties window. Search for the Paint event of the form, and double-click the row that contains the Paint event.

Modify its event handler to look like this

graphics.Form1

```
namespace graphics
{
    3 references
    public partial class Form1 : Form
    {
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void Form1_Paint(object sender, PaintEventArgs e)
        {

            Graphics grfx = e.Graphics;

            grfx.DrawString("hello sara",Font, Brushes.Black, 0,0);

        }
    }
}
```

‹/› C#

```
Graphics grfx = e.Graphics;
```

- `e.Graphics` ✅ → **this is a property**
  - It belongs to `PaintEventArgs`
  - It returns a `Graphics` object

Method Definition

```C#
private void Form1_Paint(object sender, PaintEventArgs e)
```

- This method runs **automatically when the form needs to be redrawn** (like when opening, resizing, or refreshing).
- `sender` : the object that
- `e` : contains data for pai

```C#
Graphics grfx = e.Graphics;
```

- `e.Graphics` ✅ → **this is a property**
  - It belongs to `PaintEventArgs`
  - It returns a `Graphics` object

Get Graphics Object

```C#
Graphics grfx = e.Graphics;
```

- `Graphics` is used for **drawing on the form** (text, shapes, images).
- `e.Graphics` gives you the **drawing surface** of the form.

Process: [15068] graphics.exe Lifecycle Events Thread: [10748] Main Thread Stack Frame: graphics.Form1.Form1_Paint

Form1.cs Form1.cs [Design]

graphics graphics.Form1 Form1_Paint(object sender, PaintEventArgs e)

```
3   namespace graphics
4   {
        3 references
5       public partial class Form1 : Form
6       {
            1 reference
7           public Form1()
8           {
9               InitializeComponent();
10          }
11
            1 reference
12          private void Form1_Paint(object sender, PaintEventArgs
13          {
14
15              Graphics grfx = e.Graphics;
16              string txt = string.Format("Hello every one");
17
18              grfx.DrawString(txt,Font, Brushes.Black, 300,500);
19
20          }
21      }
22  }
```

00 % No issues found

Autos

Search (Ctrl+E) Search Depth: 3

Name Value Type

Diagnostic Tools

Diagnostics session: 18 seconds

Form1

Hello every one

The DrawString() method used in Step by Step takes five arguments and has the following signature

```
public void DrawString(
    string, Font, Brush, float, float);
```

The first argument, string, is the string to be displayed.
the String.Format() method to format the string

The second argument, Font, is the font of the string
the string by using the default font of the current form through the Font property

The third parameter, Brush, is the type of brush. The Brushes enumeration provides a variety of Brush objects, each with a distinct color.

The fourth and fifth properties, float and float, specify the x and y locations for the point that marks the start of the string on the form.

Drawing Text on a Form with more formatting

Form1.cs [Design]*

graphics.Form1 | Form1_Resize(object sender, Ev

```
{
    3 references
    public partial class Form1 : Form
    {
        1 reference
        public Form1()
        {
            InitializeComponent();
            // Default Code in the constructor
            // Paint when resized
            this.ResizeRedraw = true;
        }

        1 reference
        private void Form1_Paint(object sender, PaintEventArgs e)
        {

            Graphics grfx = e.Graphics;
            String strText = String.Format("hello girls");

            PointF pt = new PointF(ClientSize.Width / 2,ClientSize.Height / 2);
            // Set the horizontal and vertical alignment
            //using StringFormat object
            StringFormat strFormat = new StringFormat();
            strFormat.Alignment = StringAlignment.Center;
            strFormat.LineAlignment = StringAlignment.Center;
            // Create Font and Brush objects
            Font fntArial = new Font("Arial", 20);
            Brush brushColor = new SolidBrush(this.ForeColor);
            // Call the DrawString method
            grfx.DrawString(strText, fntArial, brushColor, pt, strFormat);

        }
```

```
Graphics grfx = e.Graphics;
String strText = String.Format("hello girls");

PointF pt = new PointF(ClientSize.Width / 2,ClientSize.Height / 2);
// Set the horizontal and vertical alignment
//using StringFormat object
StringFormat strFormat = new StringFormat();
strFormat.Alignment = StringAlignment.Center;
```

- The code begins by calculating the center coordinates of the form. When you have the coordinates of the center, you want the string to be horizontally centred at that point.

- Calling **StringAlignment.Center** does this job. StringAlignment is an enumeration that is available in the System.Drawing namespace that specifies the location of the alignment of the text.

```
strFormat.LineAlignment = StringAlignment.Center;
```

By default, when the DrawString() method draws a string, it aligns the top of the string with its x-coordinate value. This does not make much difference if the height of the text itself is not great, but as you increase the size of the font, text begins to hang down, starting from the x-axis.

To center the text vertically within its own line, you can set the LineAlignment property of the StringFormat object to StringAlignment.Center.

🔷 4. Set text alignment

```C#
StringFormat strFormat = new StringFormat();
strFormat.Alignment = StringAlignment.Center;
strFormat.LineAlignment = StringAlignment.Center;
```

- `Alignment` → horizontal alignment
- `LineAlignment` → vertical alignment

👉 Both set to **Center**, so text is centered around the point

🔷 3. Define the center point

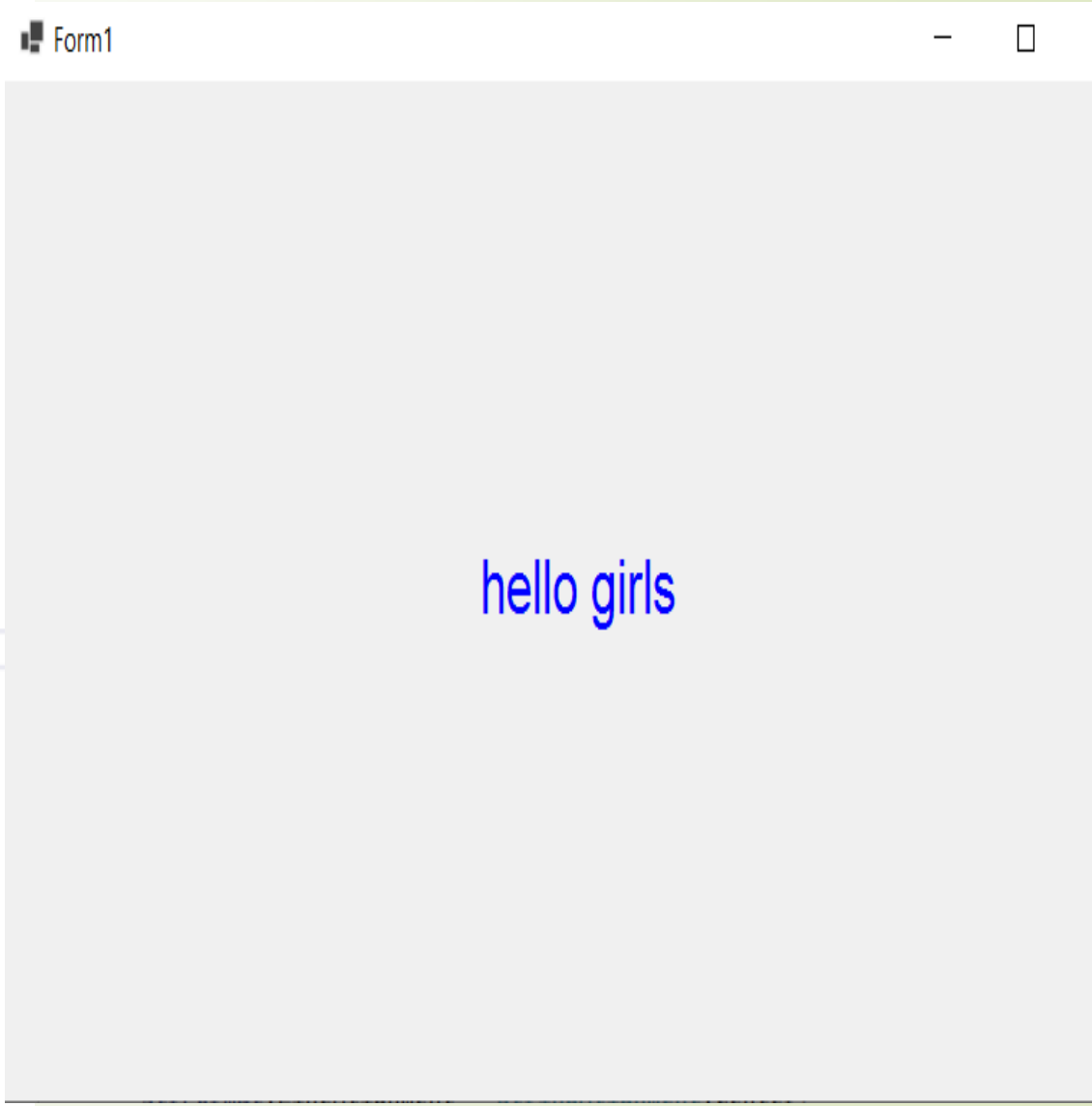
```C#
Point pt = new Point(ClientSize.Width / 2, ClientSize.Height / 2);
```

- `ClientSize.Width` → width of the form
- `ClientSize.Height` → height of the form
- Dividing by 2 → gives the **center of the form**

👉 So `pt` is the **middle point of the window**

graphics.Form1 | Form1_Paint(object s

```
{
    1 reference
    public Form1()
    {
        InitializeComponent();
        // Default Code in the constructor
        // Paint when resized
        this.ResizeRedraw = true;
    }

    1 reference
    private void Form1_Paint(object sender, PaintEventArgs e)
    {

        Graphics grfx = e.Graphics;
        String strText = String.Format("hello girls");

        PointF pt = new PointF(ClientSize.Width / 2,ClientSize.Height / 2);
        // Set the horizontal and vertical alignment
        //using StringFormat object
        StringFormat strFormat = new StringFormat();
        strFormat.Alignment = StringAlignment.Center;
        strFormat.LineAlignment = StringAlignment.Center;
        // Create Font and Brush objects
        Font fntArial = new Font("Arial", 20);
        Brush brushColor = new SolidBrush(Color.Blue);
        // Call the DrawString method
        grfx.DrawString(strText, fntArial, brushColor, pt, strFormat);

    }
```



Drawing Shapes

- The Graphics class allows you to draw various graphical shapes such as arcs, curves, pies, ellipses, rectangles, images, paths, and polygons. Some of the Graphics class methods :

Method	*Description*
`DrawArc()`	Draws an arc that represents a portion of an ellipse
`DrawBezier()`	Draws a Bézier curve defined by four points
`DrawBeziers()`	Draws a series of Bézier curves
`DrawClosedCurve()`	Draws a closed curve defined by an array of points
`DrawCurve()`	Draws a curve defined by an array of points
`DrawEllipse()`	Draws an ellipse defined by a bounding rectangle specified by a pair of coordinates, a height, and a width
`DrawIcon()`	Draws the image represented by the specified `Icon` object at the given coordinates
`DrawImage()`	Draws an `Image` object at the specified location, preserving its original size
`DrawLine()`	Draws a line that connects the two points
`DrawLines()`	Draws a series of line segments that connect an array of points
`DrawPath()`	Draws a `GraphicsPath` object

SOME IMPORTANT DRAWING METHODS OF THE Graphics CLASS

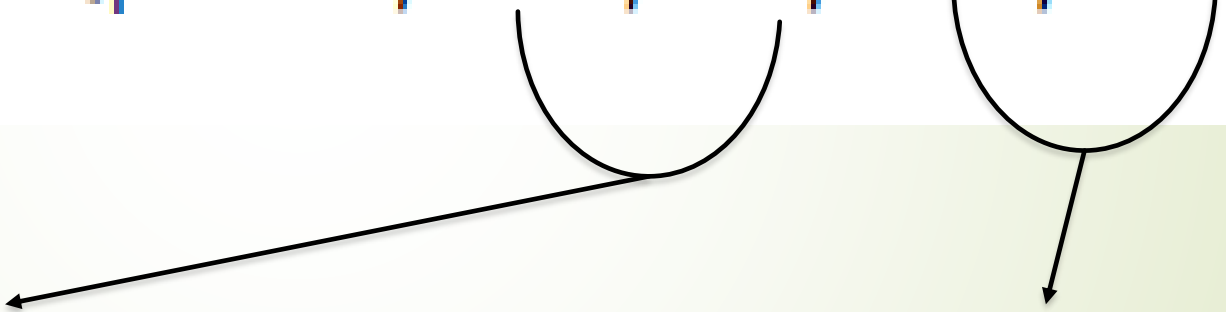
`DrawPie()`	Draws a pie shape defined by an ellipse and two radial lines
`DrawPolygon()`	Draws a polygon defined by an array of points
`DrawRectangle()`	Draws a rectangle specified by a point, a width, and a height
`DrawRectangles()`	Draws a series of rectangles
`DrawString()`	Draws the given text string at the specified location, with the specified `Brush` and `Font` objects

You can call Draw methods of the Graphics class to draw various graphic shapes.

```
        this.ResizeRedraw = true;
    }

    1 reference
    private void Form1_Paint(object sender, PaintEventArgs e)
    {

        Graphics grfx = e.Graphics;
        // Set the Smoothing mode
        // to SmoothingMode.AntiAlias
        grfx.SmoothingMode = SmoothingMode.AntiAlias;
        // Create Pen objects
        Pen penYellow = new Pen(Color.Blue, 20);
        Pen penRed = new Pen(Color.Red, 10);
        // Call Draw methods
        grfx.DrawLine(Pens.Black, 20, 130, 250, 130);
        grfx.DrawEllipse(penYellow, 20, 10, 100, 100);
        grfx.DrawRectangle(penRed, 150, 10, 100, 100);

    }
```

Output

```
// Call Draw methods
grfx.DrawLine(Pens.Black, 20, 130, 250, 130);
grfx.DrawEllipse(penYellow, 20, 10, 100, 100);
grfx.DrawRectangle(penRed, 150, 10, 100, 100);
```



The second and third arguments are the x- and y-coordinates of the upper-left corner where the desired shape is to be drawn

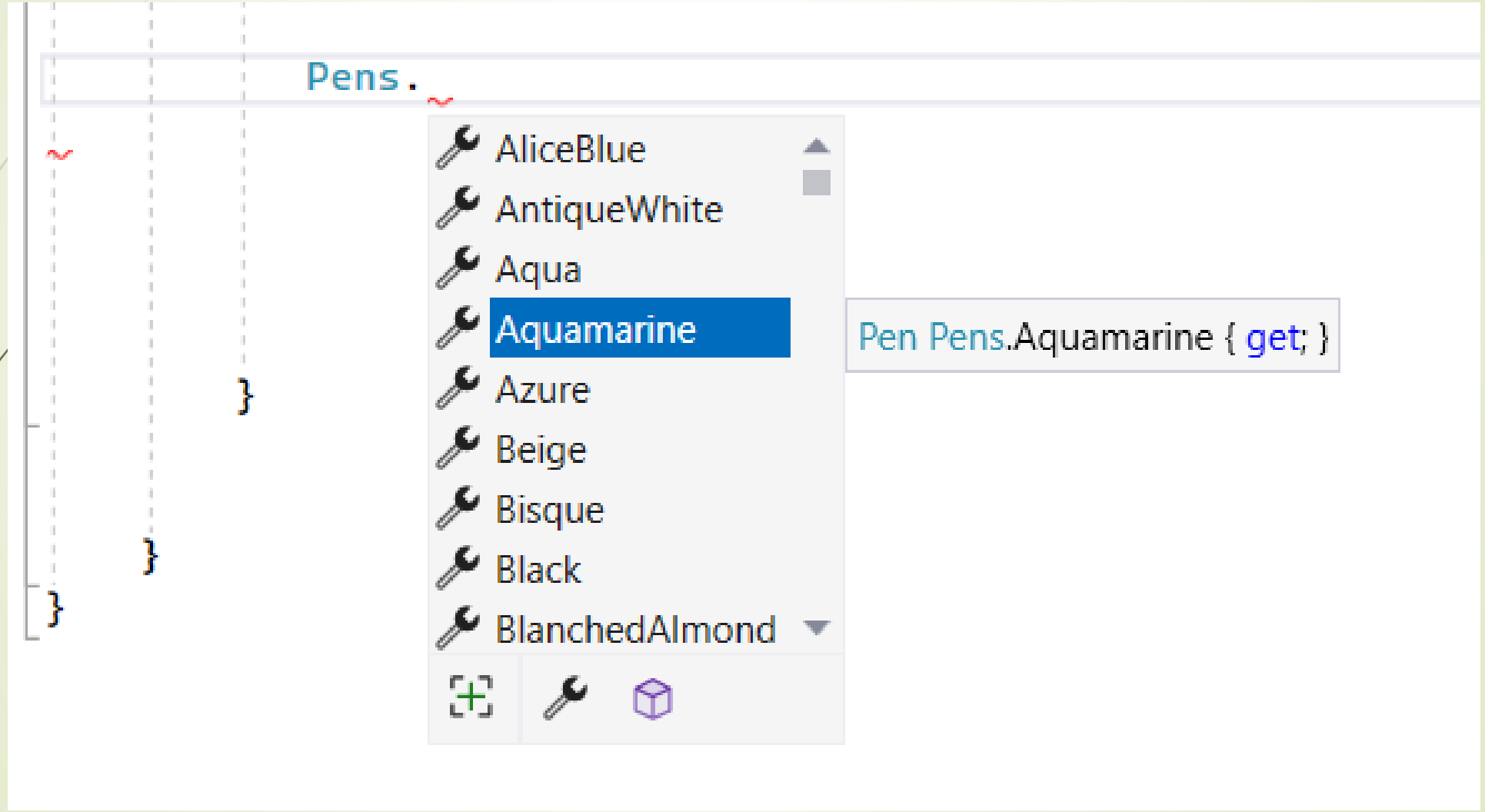
The fourth and fifth parameters indicate the width and height of the desired shape to be drawn. In the case of DrawLine, these indicate the x- and y-coordinates of the ending point of the line drawn

Pen-related classes in the System.Drawing namespace

NAMESPACE

Class	*Description*
Pen	Defines an object used to draw lines and curves.
Pens	Provides 140 static properties, each representing a pen of a color supported by Windows Forms.

Pens class with static properties colors



```
// Set the Smoothing mode
// to SmoothingMode.AntiAlias
grfx.SmoothingMode = SmoothingMode.AntiAlias;
```

The SmoothingMode property specifies the quality of rendering.

The Windows Forms library supports antialiasing, which produces text and graphics that appear to be.

C#

```
grfx.SmoothingMode = SmoothingMode.AntiAlias;
```

Explanation:

- `grfx`
  This is a `Graphics` object used to draw shapes, lines, text, etc.
- `SmoothingMode`
  A property that controls how graphics are rendered (especially edges of shapes).
- `SmoothingMode.AntiAlias`
  Enables **anti-aliasing**, which makes drawn shapes look smoother by reducing jagged edges.

What anti-aliasing does:

When drawing lines or shapes on a pixel grid, edges can look "jagged" (like stairs). Anti-aliasing softens these edges by blending colors at the borders.

```
        Graphics grfx = e.Graphics;
        // Set the Smoothing mode
        // to SmoothingMode.AntiAlias
        grfx.SmoothingMode = SmoothingMode.
        // Create Pen objects
        Pen penYellow = new Pen(Color.Blue,
        Pen penRed = new Pen(Color.Red, 10)
        // Call Draw methods
        grfx.DrawLine(Pens.Black, 20, 130,
        grfx.DrawEllipse(penYellow, 20, 10,
        grfx.DrawRectangle(penRed, 150, 10,
    }
```

- ★ None
- ★ Invalid
- AntiAlias
- Default
- HighQuality
- HighSpeed
- Invalid
- None

SmoothingMode ENUMERATION members

Member Name	*Description*
`AntiAlias`	Specifies an antialiased rendering.
`Default`	Specifies no antialiasing. Same as `None`.
`HighQuality`	Specifies a high-quality, low-performance rendering. Same as `AntiAlias`.
`HighSpeed`	Specifies a high-performance, low-quality rendering. Same as `None`.
`Invalid`	Specifies an invalid mode, raises an exception.
`None`	Specifies no antialiasing.

Graphics class also provides a variety of Fill methods, you can use these methods to draw a solid shape on a form

Fill Methods of the Graphics Class

Method Name	*Description*
`FillClosedCurve()`	Fills the interior of a closed curve defined by an array of points
`FillEllipse()`	Fills the interior of an ellipse defined by a bounding rectangle
`FillPath()`	Fills the interior of a `GraphicsPath` object
`FillPie()`	Fills the interior of a pie section defined by an ellipse and two radial lines
`FillPolygon()`	Fills the interior of a polygon defined by an array of points

Method Name	*Description*
`FillRectangle()`	Fills the interior of a rectangle specified by a point, a width, and a height
`FillRectangles()`	Fills the interiors of a series of rectangles
`FillRegion()`	Fills the interior of a `Region` object

Form1.cs [Design]

graphics.Form1 | Form1_Paint(object sender, PaintEventArgs e)

```
{
    InitializeComponent();
    // Default Code in the constructor
    // Paint when resized
    this.ResizeRedraw = true;
}

1 reference
private void Form1_Paint(object sender, PaintEventArgs e)
{
    Graphics grfx = e.Graphics;
    // Create Brush objects
    Brush brushRed = new SolidBrush(Color.Red);
    Brush brushYellow = new SolidBrush(Color.FromArgb(200, Color.Yellow));
    // Call Fill methods
    grfx.FillEllipse(brushRed, 20, 10, 80, 100);
    grfx.FillRectangle(brushYellow, 60, 50, 100, 100);

}
```

Diagnostic Tools

Diagnostics session: 55 seconds

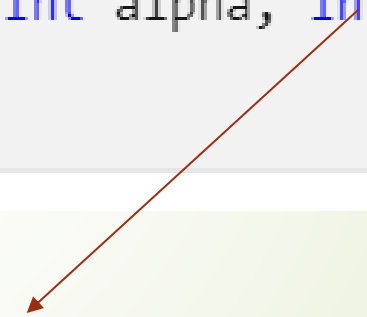
50s

Events

Form1

C# Copy

```
public static System.Drawing.Color FromArgb (int alpha, int red,
int green, int blue);
```



Value from 0 to 255

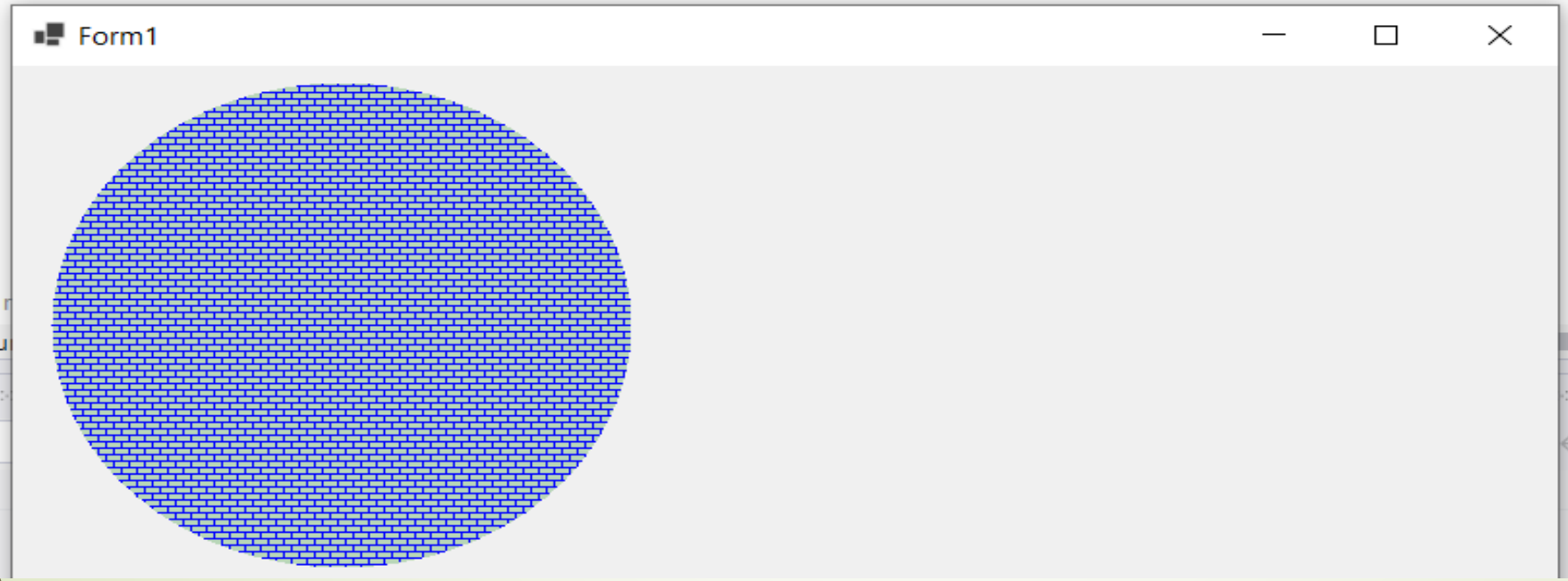
- The syntax of the Fill methods is somewhat similar to that of corresponding Draw methods. The only difference is that the Fill methods use a Brush object to fill a drawing object with a color.

Types of Brushes in the System.Drawing and System.Drawing.Drawing2D Namespaces

Class	*Description*
`Brush`	Is an abstract base class that is used to create brushes such as `SolidBrush`, `TextureBrush`, and `LinearGradientBrush`. These brushes are used to fill the interiors of graphical shapes such as rectangles, ellipses, pies, polygons, and paths.
`Brushes`	Provides 140 static properties, one for the name of each color supported by Windows forms.
`HatchBrush`	Allows you to fill the region by using one pattern from a large number of patterns available in the `HatchStyle` enumeration.
`LinearGradientBrush`	Is used to create two-color gradients and multicolor gradients. By default the gradient is a linear gradient that moves from one color to another color along the specified line.

graphics.Form1 | Form1_Paint(object sender, PaintEventArgs e)

```
        this.ResizeRedraw = true;
    }

    1 reference
    private void Form1_Paint(object sender, PaintEventArgs e)
    {

        Graphics grfx = e.Graphics;
        // Create a HatchBrush object
        // Call FillEllipse method by passing
        // the created HatchBrush object
        HatchBrush hb = new HatchBrush(HatchStyle.HorizontalBrick,Color.Blue, Color.FromArgb(59, Color.Green));
        grfx.FillEllipse(hb, 20, 10, 300, 300);
```



No issues fou

Ln: 24 Ch:

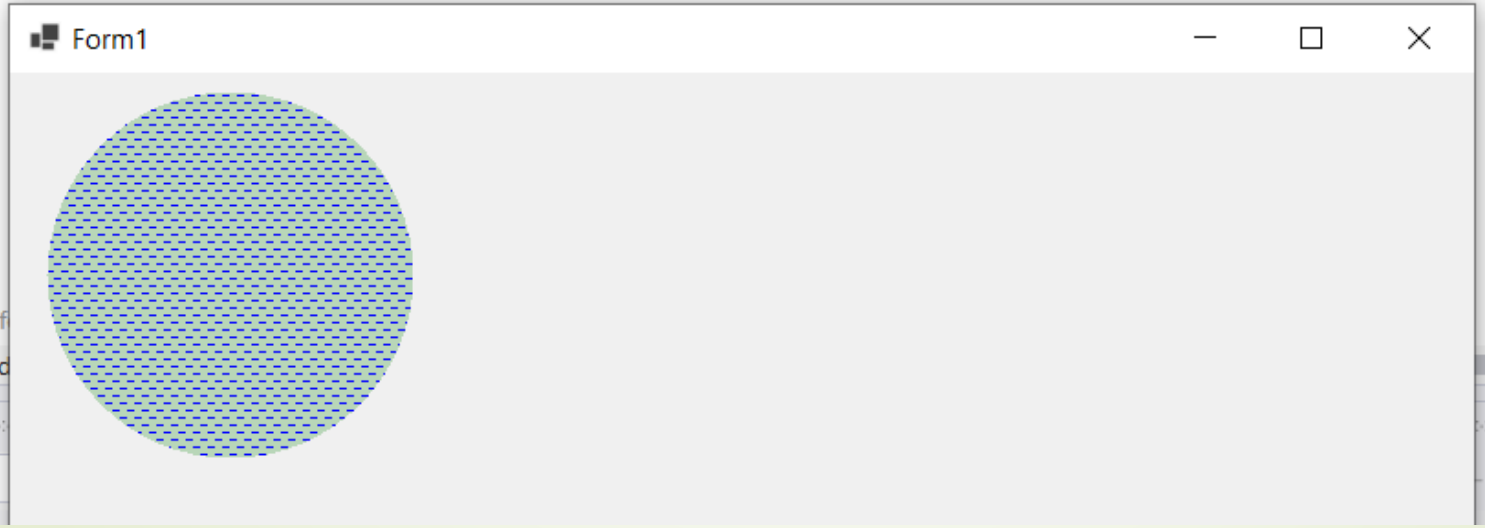
View all Threads Show

Using HatchBrush, you filled the ellipse with the HatchStyle named HorizontalBrick.

Example2: Using HatchBrush

graphics.Form1 | Form1_Paint(object sender, PaintEventArgs e)

```
        this.ResizeRedraw = true;
    }

    1 reference
    private void Form1_Paint(object sender, PaintEventArgs e)
    {

        Graphics grfx = e.Graphics;
        // Create a HatchBrush object
        // Call FillEllipse method by passing
        // the created HatchBrush object
        HatchBrush hb = new HatchBrush(HatchStyle.DashedHorizontal,Color.Blue, Color.FromArgb(59, Color.Green));
        grfx.FillEllipse(hb, 20, 10, 200, 200);

    }
```



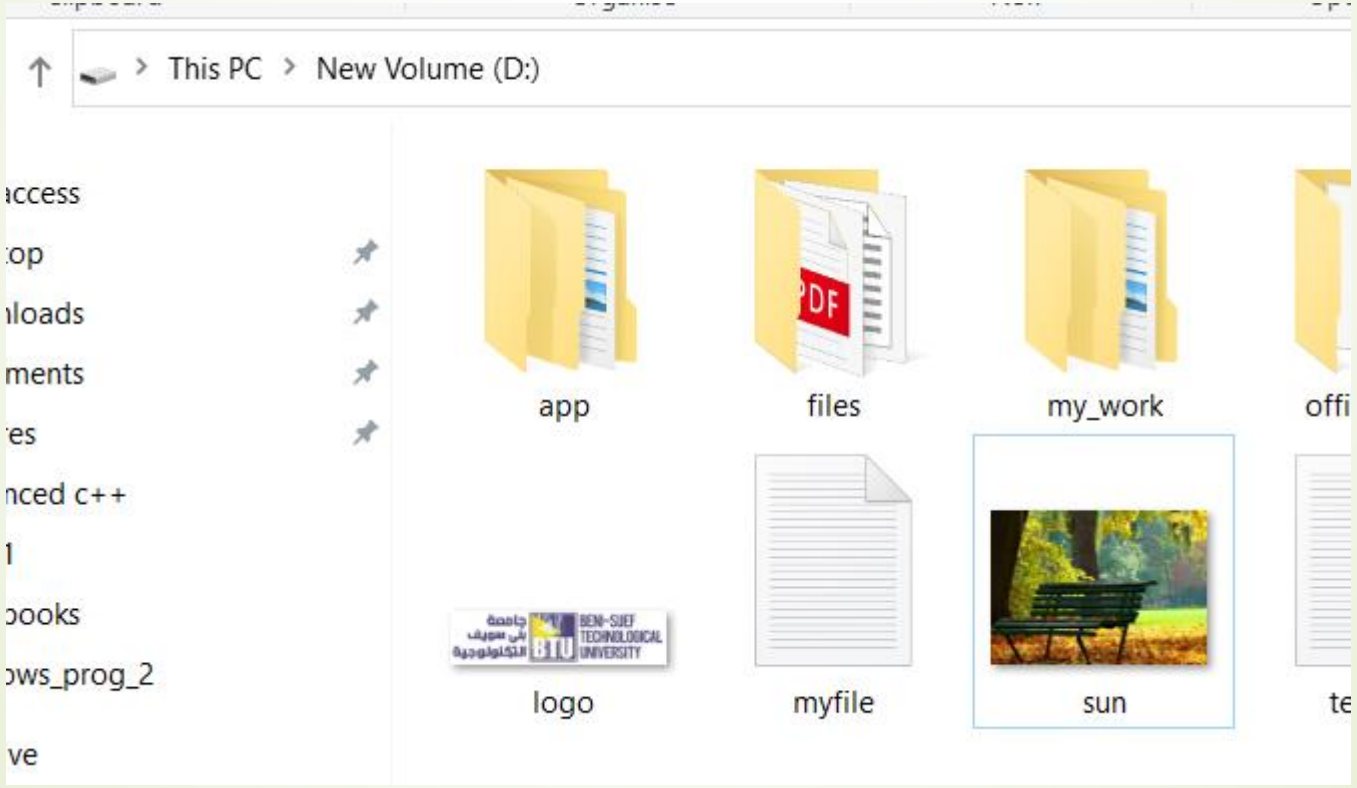
No issues found | Ln: 24 | Ch: 48

View all Threads | Show E

The TextureBrush class

Example

- First we will use this image file :

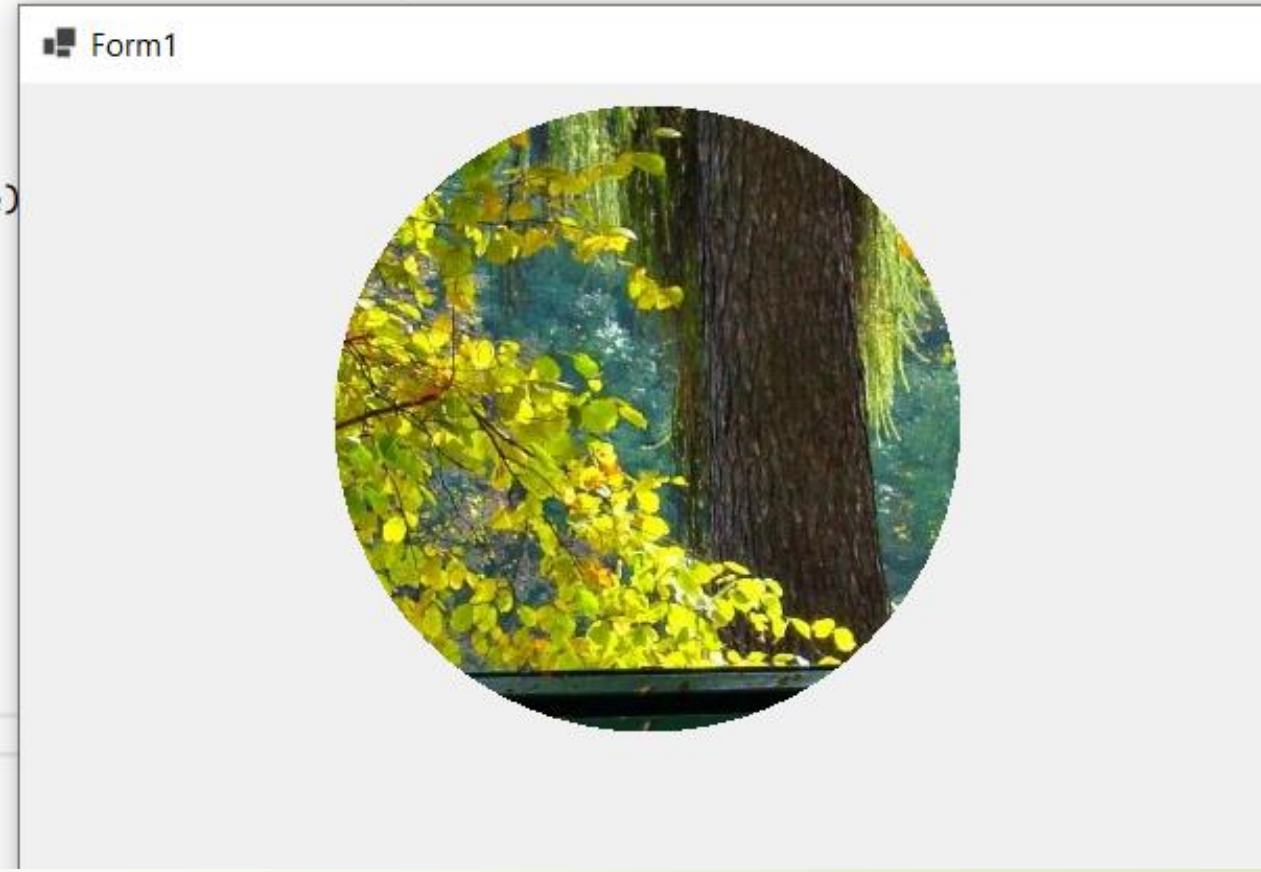


Example3: The TextureBrush class

m1.cs [Design]

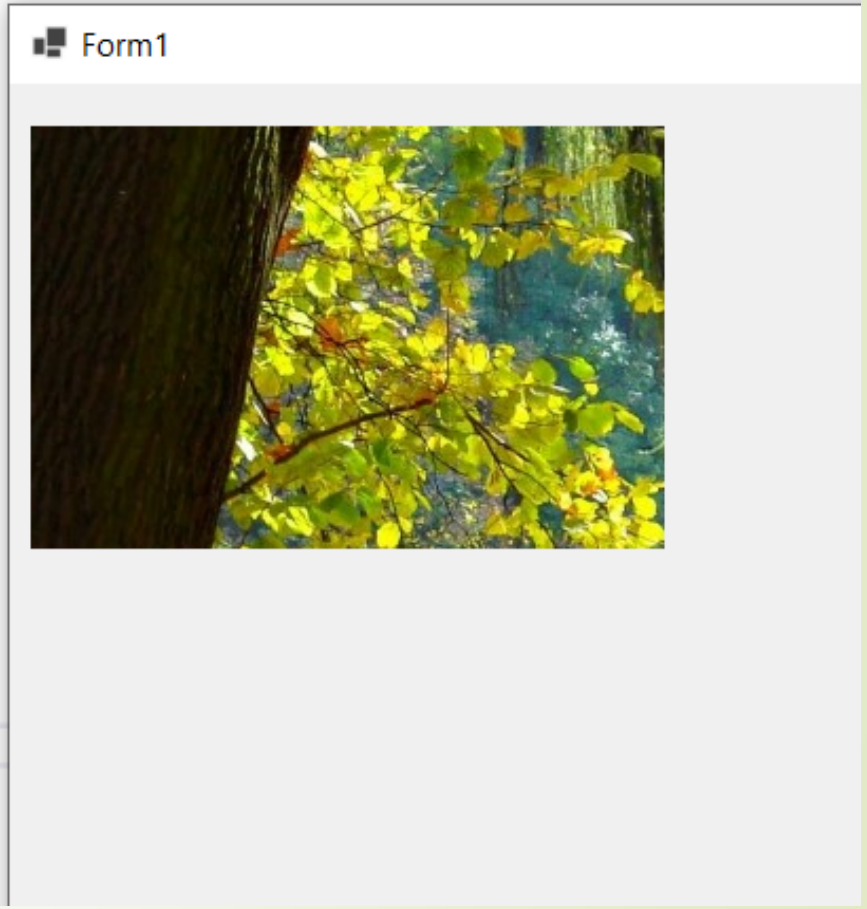
graphics.Form1 | Form1_Paint(object sender, PaintEventArgs e)

```
        this.ResizeRedraw = true;
    }

    1 reference
    private void Form1_Paint(object sender, PaintEventArgs e)
    {

        Graphics grfx = e.Graphics;
        // Create a TextureBrush object
        // Call FillEllipse method by passing the
        // created TextureBrush object
        Image img = new Bitmap(@"D:\sun.png");
        TextureBrush tb = new TextureBrush(img);
        grfx.FillEllipse(tb, 150, 10, 300, 300);



    }
```



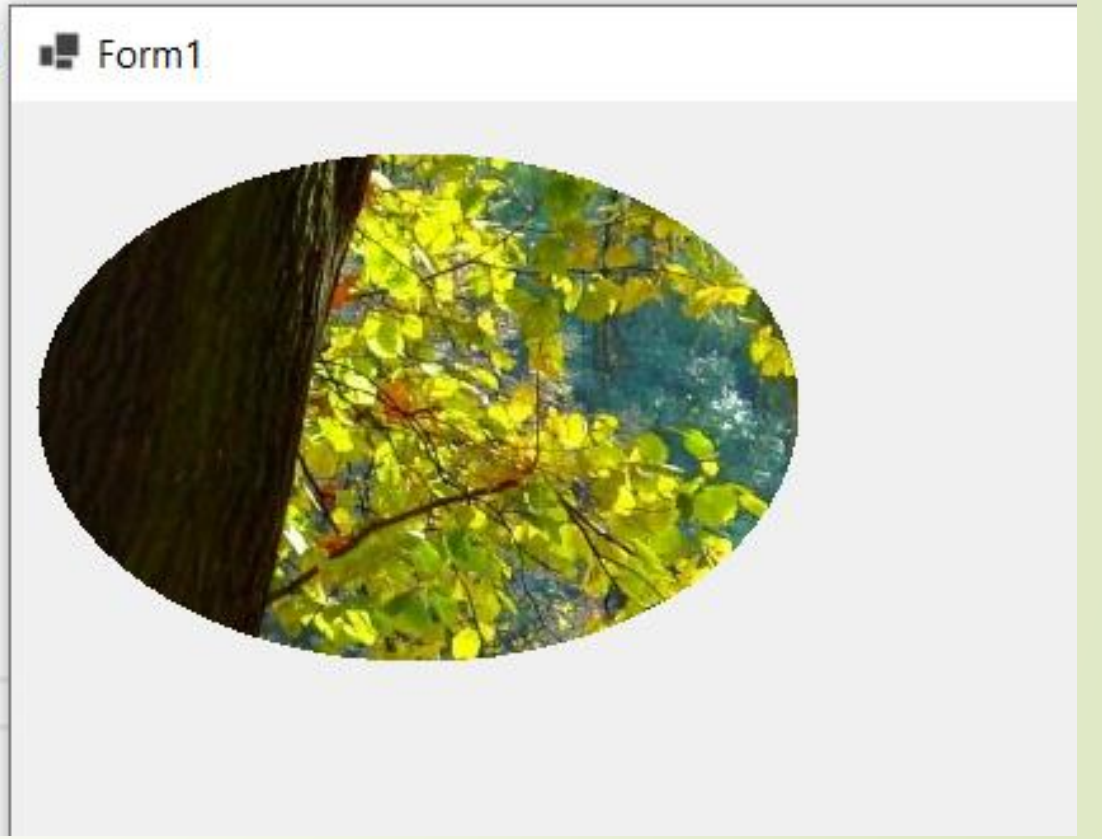
Using HatchBrush, you filled the ellipse with the HatchStyle named HorizontalBrick. The TextureBrush class uses an image to fill the interior of a shape

```
s                                   graphics.Form1                    Form1_Paint(object sender, Pair
13          this.ResizeRedraw = true;
14      }
15
            1 reference
16      private void Form1_Paint(object sender, PaintEventArgs e)
17      {
18
19          Graphics grfx = e.Graphics;
20          // Create a TextureBrush object
21          // Call FillEllipse method by passing the
22          // created TextureBrush object
23          Image img = new Bitmap(@"D:\sun.png");
24          TextureBrush tb = new TextureBrush(img);
25          grfx.FillRectangle(tb, 10, 20, 300, 200);
26
27
28
29
30      }
31
            1 reference
```



graphics.Form1 | Form1_Paint(object sender, Paint

```
    this.ResizeRedraw = true;
}

1 reference
private void Form1_Paint(object sender, PaintEventArgs e)
{

    Graphics grfx = e.Graphics;
    // Create a TextureBrush object
    // Call FillEllipse method by passing the
    // created TextureBrush object
    Bitmap img = new Bitmap(@"D:\sun.png");
    TextureBrush tb = new TextureBrush(img);
    grfx.FillEllipse(tb, 10, 20, 300, 200);



}
```

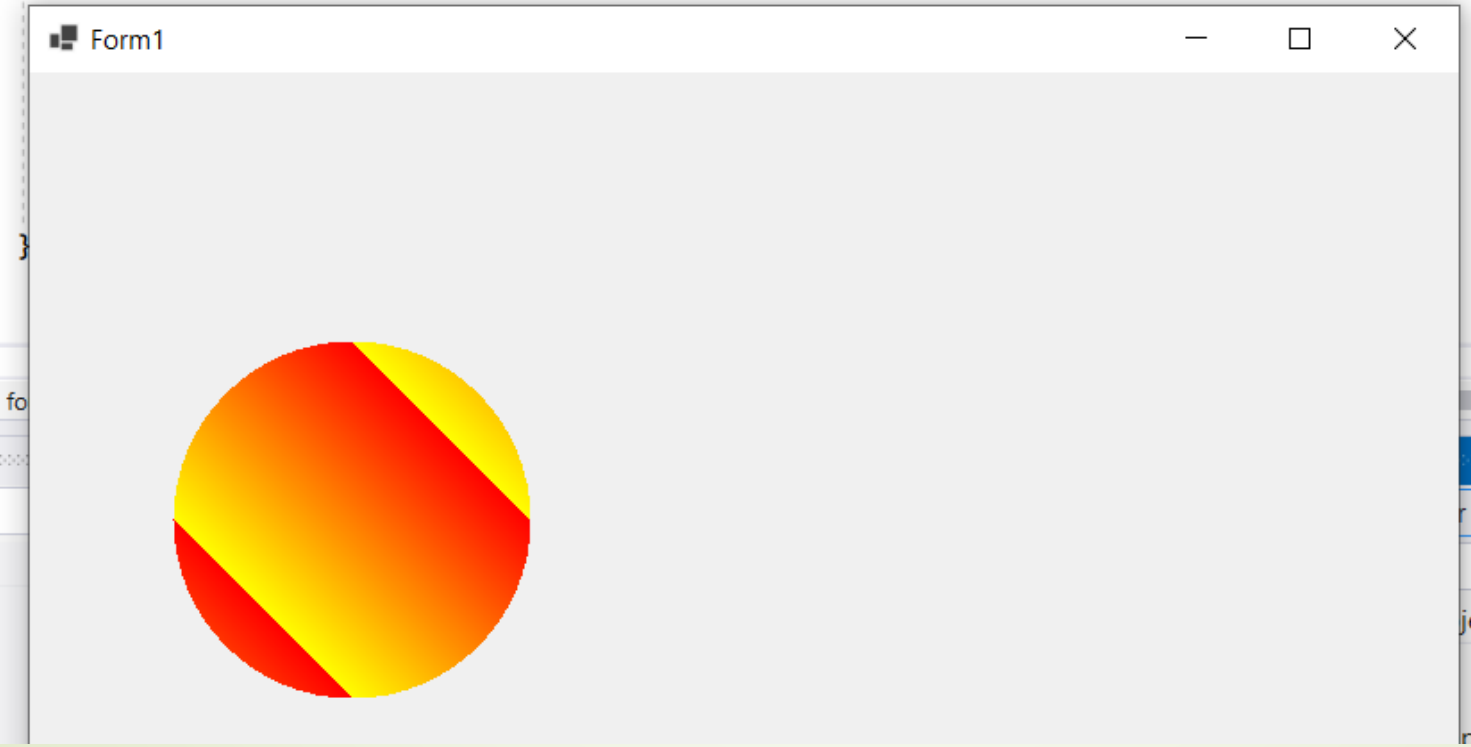


LinearGradientBrush class

Form1.cs [Design]

graphics.Form1 | InitializeComponent()

```
1 reference
private void Form1_Paint(object sender, PaintEventArgs e)
{

    Graphics grfx = e.Graphics;
    // Create a LinearGradientBrush object
    // Call FillEllipse method by passing
    // the created LinearGradientBrush object
    LinearGradientBrush lb = new LinearGradientBrush(new Rectangle(80, 150, 100, 100), Color.Red, Color.Yellow,
    LinearGradientMode.BackwardDiagonal);
    grfx.FillEllipse(lb, 80, 150, 200, 200);
```



No issues fo

Ln: 37 Ch: 6

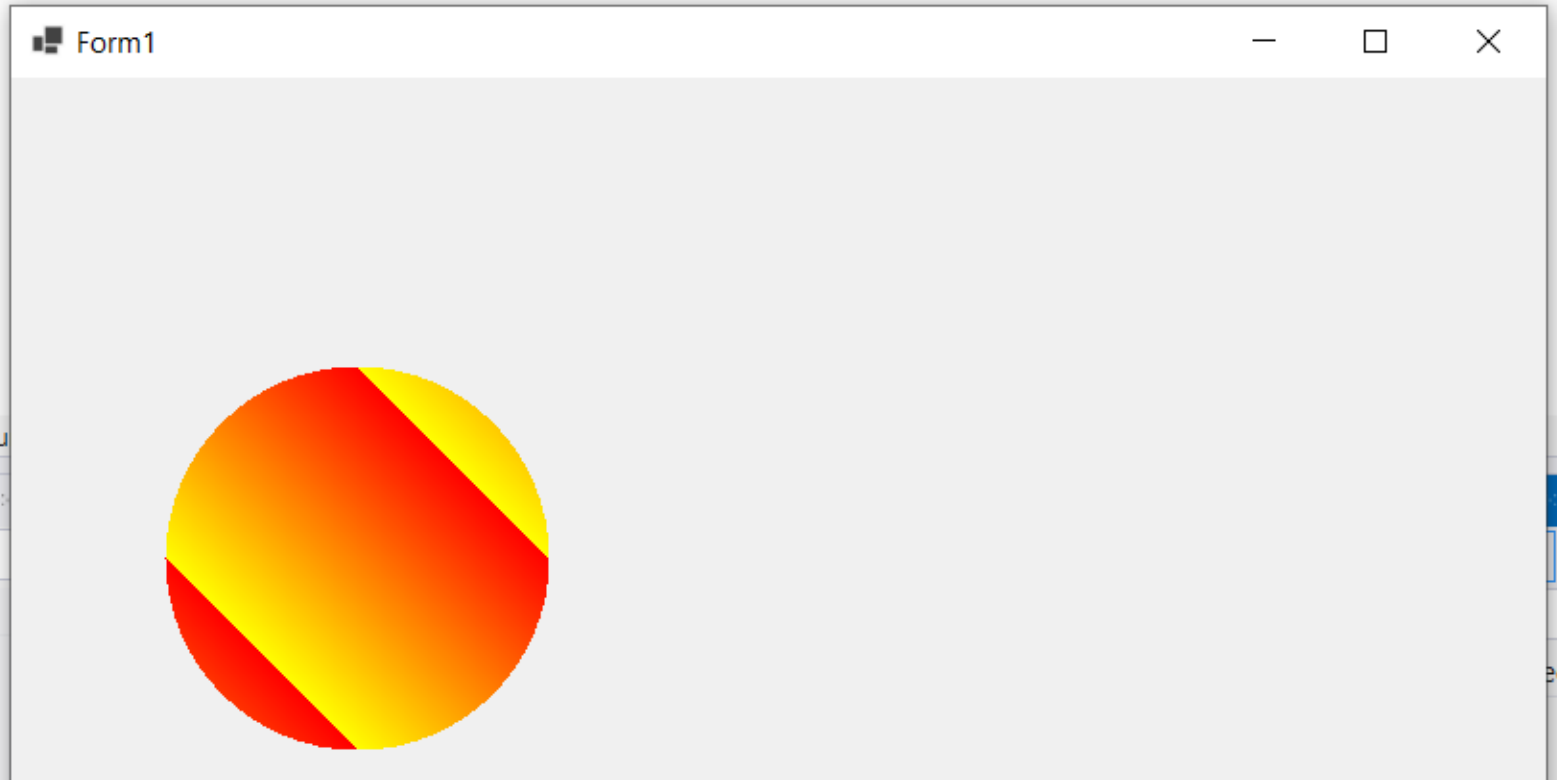
0 Warnings | 0 of 1 Message

ject | File

phics | Form1.Designer.cs

graphics.Form1 | Form1_Paint(object

```
1 reference
private void Form1_Paint(object sender, PaintEventArgs e)
{

    Graphics grfx = e.Graphics;
    // Create a LinearGradientBrush object
    // Call FillEllipse method by passing
    // the created LinearGradientBrush object
    Rectangle rect = new Rectangle(80, 150, 100, 100);
    LinearGradientBrush lb = new LinearGradientBrush(rect, Color.Red, Color.Yellow,
    LinearGradientMode.BackwardDiagonal);
    grfx.FillEllipse(lb, 80, 150, 200, 200);

}
```



LinearGradientMode Enumeration Values

Member Name	*Description*
`BackwardDiagonal`	Specifies a gradient from upper-right to lower-left
`ForwardDiagonal`	Specifies a gradient from upper-left to lower-right
`Horizontal`	Specifies a gradient from left to right
`Vertical`	Specifies a gradient from top to bottom

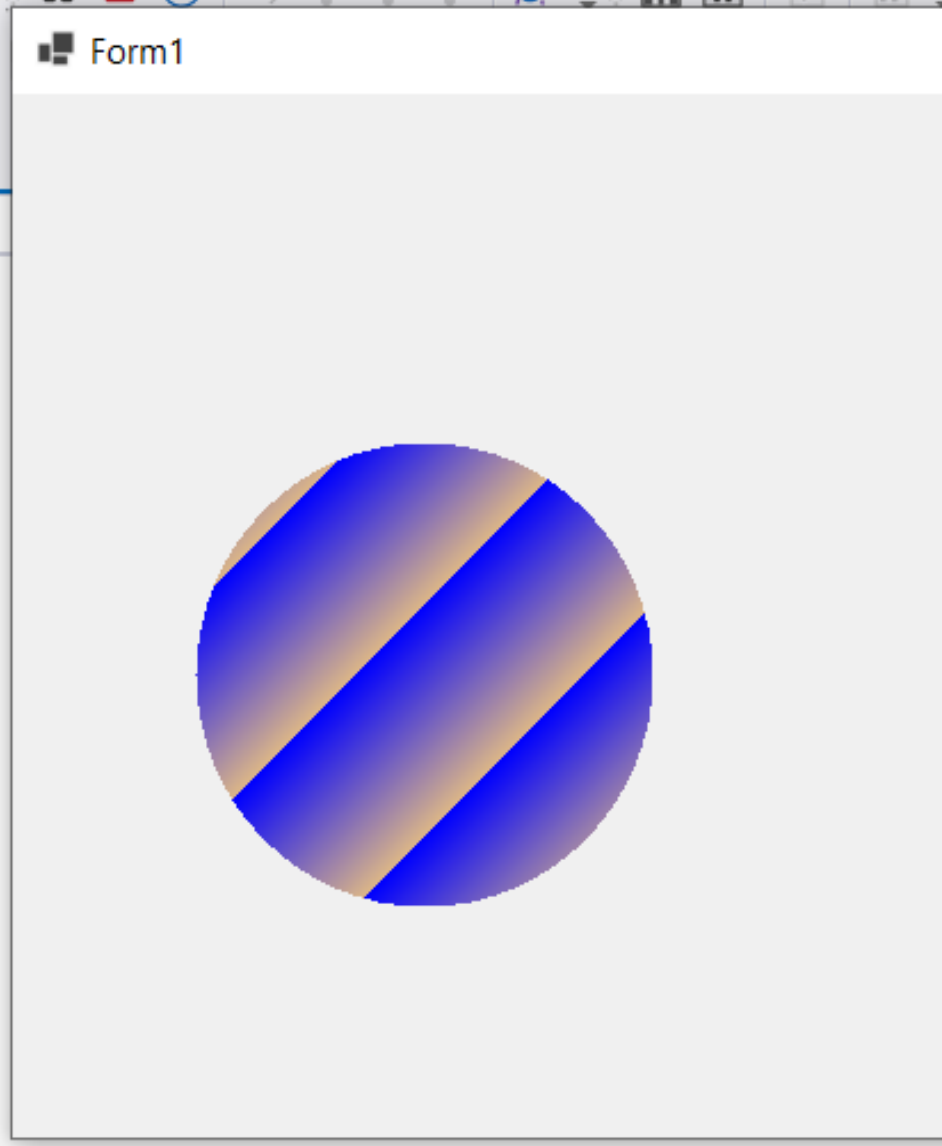
Debug | Any CPU | Continue

24180] WinFormsApp11.exe | Lifecycle Events | Thread: [8756] Main Thread

Form1.cs [Design]

sApp11 | WinFormsApp11.Form1

```
namespace WinFormsApp11
{
    3 references
    public partial class Form1 : Form
    {
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void Form1_Paint(object sender, PaintEventArgs e)
        {
            Graphics grfx = e.Graphics;
            // Create a LinearGradientBrush object
            // Call FillEllipse method by passing
            // the created LinearGradientBrush object
            Point pt = new Point(50, 50);
            Point pt2 = new Point(100, 100);
            LinearGradientBrush lb = new LinearGradientBrush(pt, pt2, Color.Blue, Color.BurlyWood);
            grfx.FillEllipse(lb, 80, 150, 200, 200);
```



Working with Images

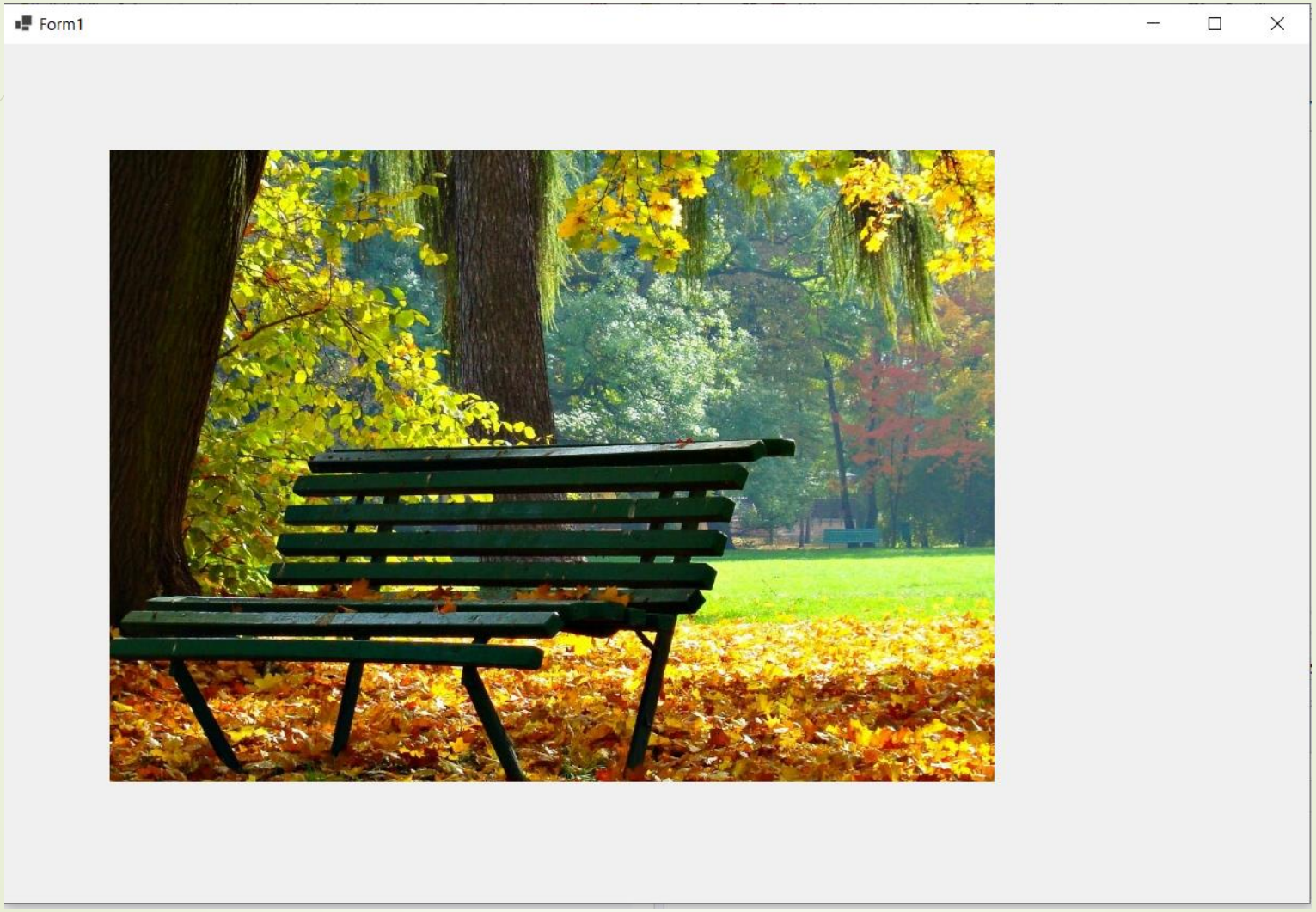
- The System.Drawing. Image class provides the basic functionality for working with images. However, the Image class is **abstract**, which means you can create an instance of it in your class. Instead of using the Image class directly, you can use the following classes that implement the Image class functionality.

- **`Bitmap`** This class is used to work with graphic files that store information in pixel-based data such as BMP, GIF, and JPEG formats.
- **`Icon`** This class creates a small bitmap that represents an Windows icon.
- **`MetaFile`** This class contains embedded bitmaps and/or sequences of binary records that represent a graphical operation such as drawing a line.

Draw image on the form in specific point with the width and height of the image

```
10          }
11
            1 reference
12          private void Form1_Paint(object sender, PaintEventArgs e)
13          {
14              Graphics grfx = e.Graphics;
15              Bitmap img = new Bitmap("D:\\sun.png");
16
17              grfx.DrawImage(img,100,100,img.Width,img.Height);
18
19
20          }
21      }
22  }
```

Output

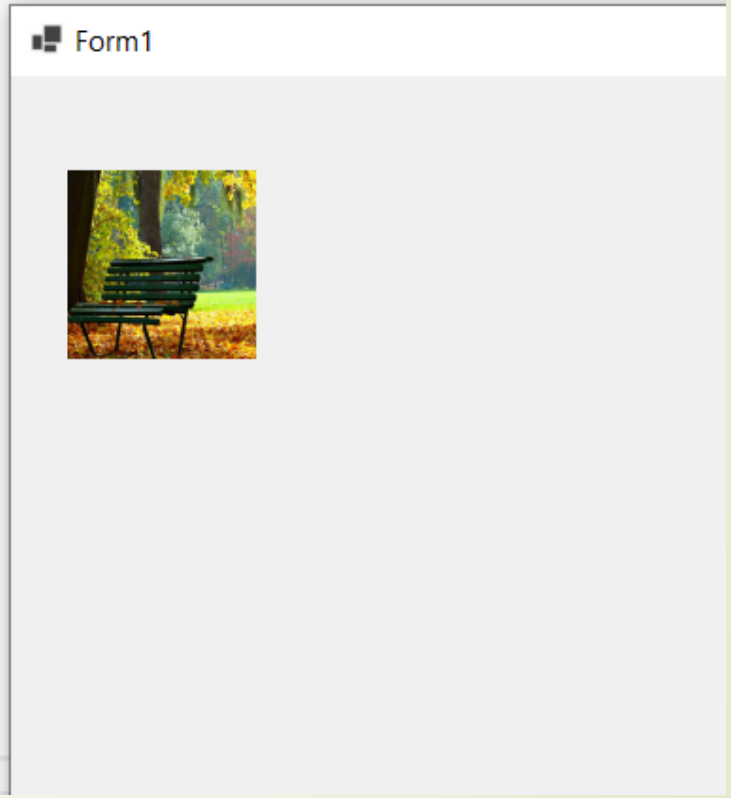


You can draw an image on a rectangle with specified size

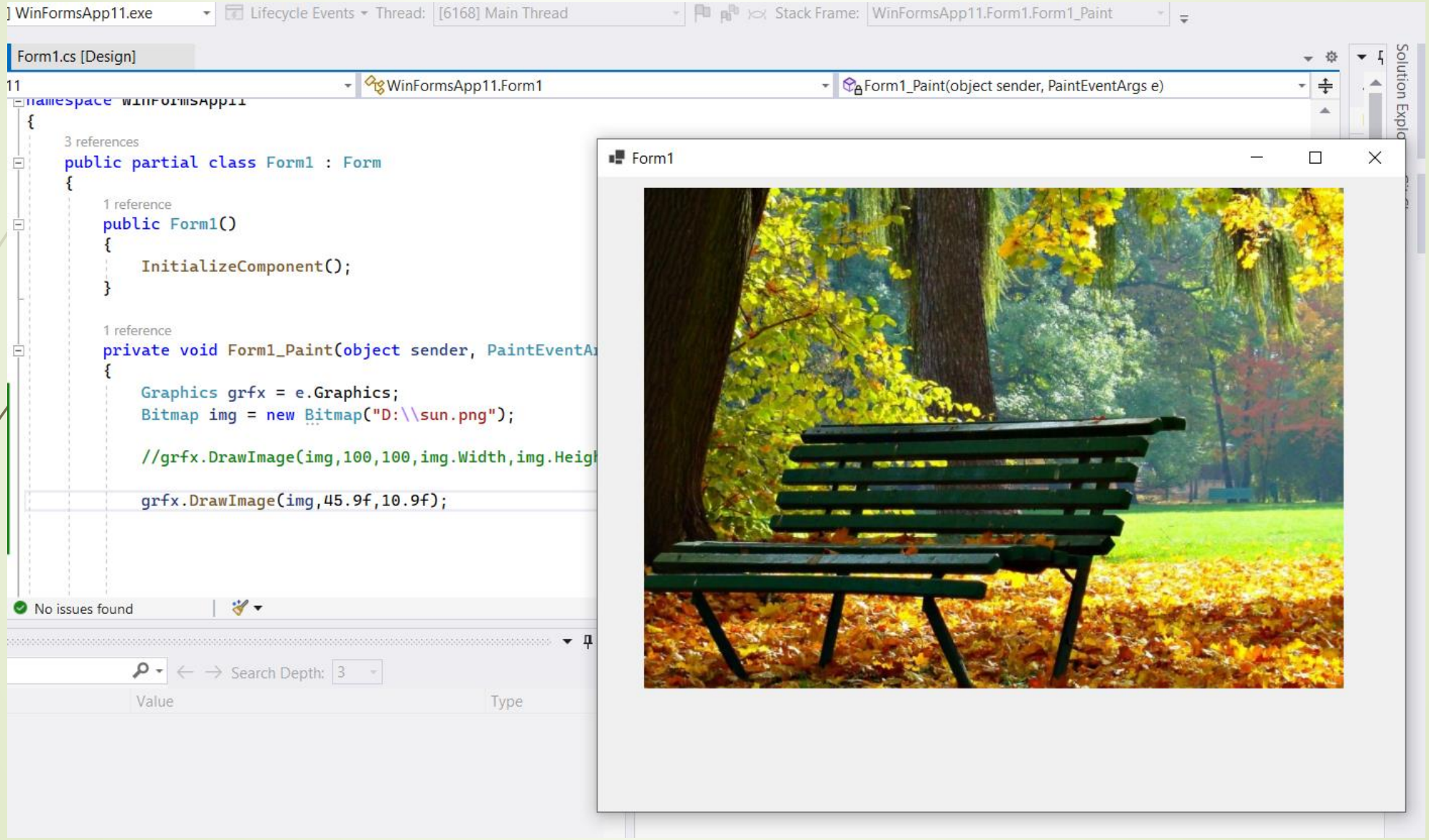
Form1.cs [Design]

WinFormsApp11.Form1 | InitializeComponent()

```
namespace WinFormsApp11
{
    3 references
    public partial class Form1 : Form
    {
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void Form1_Paint(object sender, PaintEventArgs e)
        {
            Graphics grfx = e.Graphics;
            Bitmap img = new Bitmap("D:\\sun.png");

            grfx.DrawImage(img,new Rectangle(30,50,100,100));


        }
    }
}
```



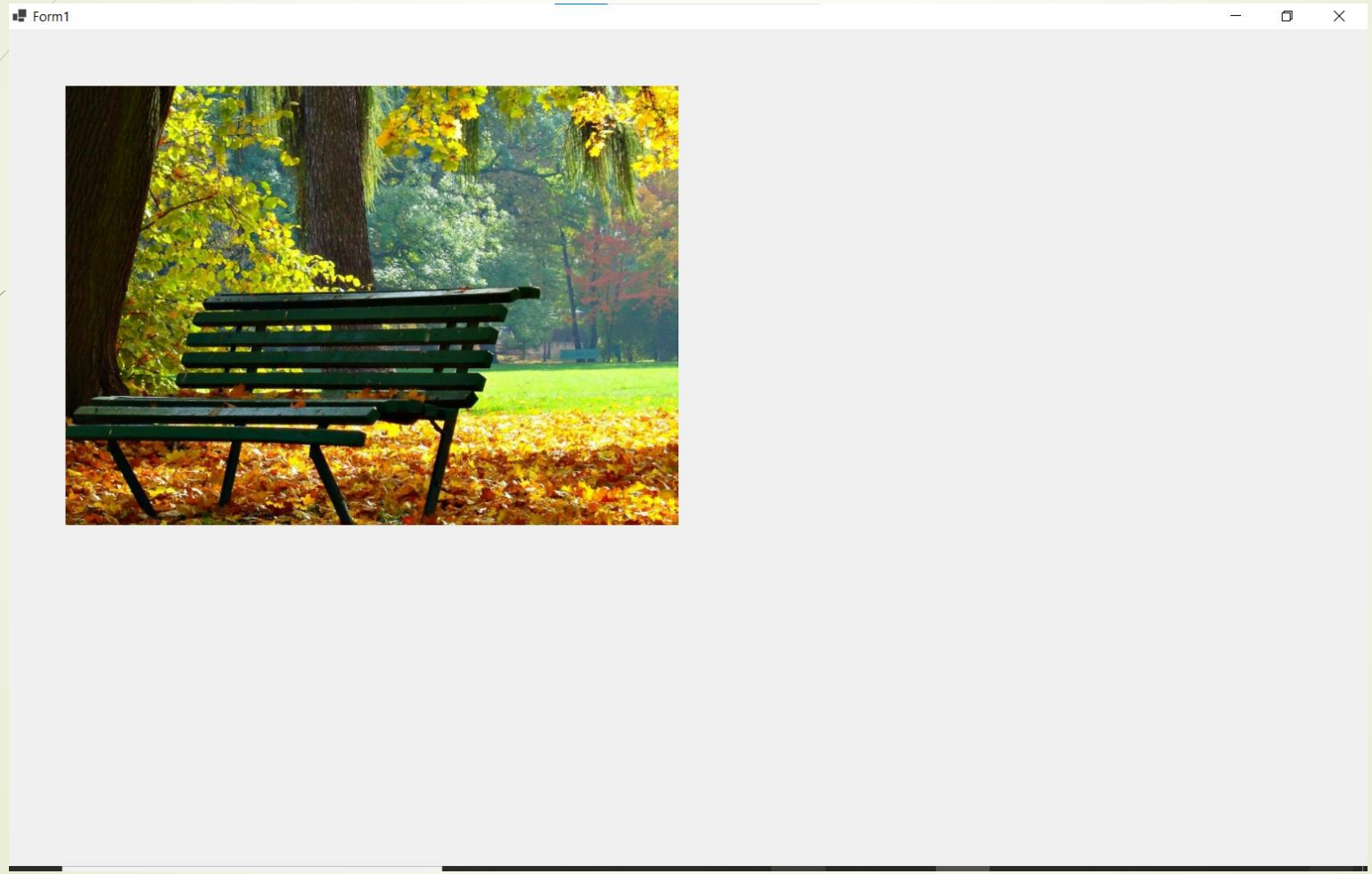
Draw image in float points(x , y) in the form



Put the image on the form top left point

```
1 reference
private void Form1_Paint(object sender, PaintEventArgs e)
{
    Graphics grfx = e.Graphics;
    Bitmap img = new Bitmap("D:\\sun.png");

    Point pt = new Point(Top, Left);

    grfx.DrawImage(img,pt);



}
}
```

Output



WinFormsApp11.Form1 | InitializeComponent()

```
        InitializeComponent();
    }

    1 reference
    private void Form1_Paint(object sender, PaintEventArgs e)
    {
        Graphics grfx = e.Graphics;
        Bitmap img1 = new Bitmap("D:\\sun.png");
        Bitmap img2 = new Bitmap("D:\\logo.jpg");

        grfx.DrawImage(img1, new Rectangle(100, 100, 200, 200));
        grfx.DrawImage(img2, new Rectangle(100, 100, 150, 150));
```

Form1


```
1 reference
private void Form1_Paint(object sender, PaintEventArgs e)
{
    Graphics grfx = e.Graphics;
    Image img1 = new Bitmap("D:\\sun.png");
    Bitmap img2 = new Bitmap("D:\\logo.jpg");

    grfx.DrawImage(img1, new Rectangle(100, 100, 200, 200));
    grfx.DrawImage(img2, new Rectangle(100, 100, 150, 150));
```

Image class is abstract so we can use it by the object of Bitmap class (subclass from it)

Thank you

References

Developing and Implementing Windows Based Applications with Microsoft Visual C# .NET and Microsoft Visual Studio.